PathForge — Complete Deployment Guide

> ***Target architecture***
>
> | *Service* | *Platform* | *URL* |
> |---|---|---|
> | *Next.js frontend* | *Vercel* | `https://yourdomain.com` |
> | *Express backend* | *Railway* | `https://pathforge-backend.up.railway.app` |
> | *Flask ML API* | *Railway* | `https://pathforge-ml.up.railway.app` |
> | *PostgreSQL* | *Neon* | *connection string only* |

Prerequisites — Accounts to Create

Before you start, sign up for:

1. **GitHub** — https://github.com (your code must be in a GitHub repo)
2. **Vercel** — https://vercel.com (free Hobby plan is sufficient)
3. **Railway** — https://railway.app (free trial, then $5/month Hobby plan)
4. **Neon** — https://neon.tech (free tier, 0.5 GB storage)

Step 0 — Code Changes (Already Applied)

These four changes have already been made to the repo. No action required — just verify they are present before pushing.

0a. Flask — dynamic PORT (required by Railway)

File: `Path-Forge API/app.py` — bottom of file

```
if __name__ == "__main__":
    import os
    port = int(os.environ.get("PORT", 8000))
    app.run(debug=False, host="0.0.0.0", port=port)
```

0b. Express — restricted CORS origin

File: `PathForgeBackend/src/index.ts`

```
app.use(cors({
  origin: process.env.FRONTEND_URL || "http://localhost:3000",
  credentials: true,
}));
```

0c. Procfile for Express backend

File: `PathForgeBackend/Procfile`

```
web: npm run build && npx prisma migrate deploy && node dist/index.js
```

0d. Procfile for Flask ML API

File: `Path-Forge API/Procfile`

```
web: python app.py
```

Step 1 — Push the Repo to GitHub

```
# From the project root
git add -A
git commit -m "chore: production-ready config (Procfiles, CORS, Flask PORT)"
git push origin main
```

If you haven't created a GitHub repo yet:

1. Go to https://github.com/new
2. Name it `PathForge_Career_Intelligence_System`, set it to **Private**
3. Follow the "...or push an existing repository" instructions shown on that page

---

Step 2 — Database: Neon (PostgreSQL)

1. Log in to https://console.neon.tech
2. Click **New Project** → name it `pathforge` → choose region **AWS / us-east-1** (or nearest to you)
3. Copy the **Connection string** — it looks like:

   ```
   postgresql://neondb_owner:<password>@ep-xxx.us-east-1.aws.neon.tech/neondb?sslmode=require
   ```

   Save this — you will use it as `DATABASE_URL` everywhere below.
4. Run Prisma migrations against Neon **from your local machine**:

   ```
   cd PathForgeBackend
   DATABASE_URL="<your-neon-connection-string>" npx prisma migrate deploy
   ```
5. Seed the database (see Step 8 for the seed command).

Step 3 — Express Backend: Railway

1. Go to https://railway.app → **New Project** → **Deploy from GitHub repo**
2. Select `PathForge_Career_Intelligence_System`
3. Railway will detect the repo root. You must change the **root directory** to `PathForgeBackend`

- Settings → **Source** → Root Directory → enter `PathForgeBackend`

4. Railway reads the `Procfile` automatically and runs: `npm run build && npx prisma migrate deploy && node dist/index.js`

5. Set the following **environment variables** (Settings → Variables):

Variable	Value
`DATABASE_URL`	your Neon connection string
`JWT_SECRET`	run `openssl rand -base64 32` and paste the output
`NEXTAUTH_SECRET`	run `openssl rand -base64 32` and paste the output (**keep this value — you will reuse it for Vercel**)
`FRONTEND_URL`	`https://yourdomain.com` (your actual domain)
`NODE_ENV`	`production`

6. Click **Deploy**. Once the build finishes, go to **Settings → Networking → Generate Domain**. Your backend URL will be something like `https://pathforge-backend.up.railway.app`.

> *Replace every occurrence of `pathforge-backend.up.railway.app` in this guide with your actual Railway domain.*

Step 4 — Flask ML API: Railway

1. Inside the same Railway project, click **+ New Service** → **GitHub Repo** → same repo

2. Set root directory to `Path-Forge API`

   - Settings → Source → Root Directory → `Path-Forge API`

3. Railway injects `PORT` automatically — no extra env vars needed for basic operation.

4. Optional env var:

Variable	Value
`PYTHON_VERSION`	`3.11`

5. Click **Deploy**. After the build, generate a domain the same way (e.g., `https://pathforge-ml.up.railway.app`).

> *Verify the Flask API is live: open `https://pathforge-ml.up.railway.app/` — you should see a JSON health response.*

Step 5 — Frontend: Vercel

1. Go to https://vercel.com/new → **Import Git Repository** → select `PathForge_Career_Intelligence_System`

2. Set **Root Directory** to `Pathforge-Frontend`
3. Framework preset: **Next.js** (auto-detected)
4. Set the following **environment variables** (Vercel dashboard → Settings → Environment Variables):

Variable	Value
`NEXT_PUBLIC_API_BASE_URL`	`https://pathforge-backend.up.railway.app`
`BACKEND_URL`	`https://pathforge-backend.up.railway.app`
`NEXT_PUBLIC_ML_API_URL`	`https://pathforge-ml.up.railway.app`
`DATABASE_URL`	your Neon connection string
`NEXTAUTH_URL`	`https://yourdomain.com` (your actual domain)
`NEXTAUTH_SECRET`	**same value** you used for the Express backend
`GOOGLE_CLIENT_ID`	from Google Cloud Console (same as local)
`GOOGLE_CLIENT_SECRET`	from Google Cloud Console (same as local)

5. Click **Deploy**. Vercel will give you a `*.vercel.app` URL — test it before attaching your custom domain.

Step 6 — Custom Domain: name.com → Vercel

6a. Add your domain in Vercel

1. Vercel dashboard → your project → **Settings → Domains**
2. Click **Add** → type `yourdomain.com` → click **Add**
3. Also add `www.yourdomain.com` if you want the `www` subdomain to work
4. Vercel will show you the DNS records to create

6b. Create DNS records in name.com

1. Log in to https://www.name.com → **My Domains** → click your domain → **Manage DNS Records**
2. Delete any existing **A records** or **CNAME records** pointing to name.com parking pages
3. Add the following records:

Type	Host	Value	TTL
`A`	`@`	`76.76.21.21`	300
`CNAME`	`www`	`cname.vercel-dns.com`	300

> *These are Vercel's standard DNS values. Vercel's dashboard may show slightly different values — use whatever Vercel instructs for your specific domain.*

4. Save the records. DNS propagation takes **5–30 minutes** (up to 48 hours in rare cases).

5. Once propagated, Vercel automatically provisions an SSL certificate. The padlock icon will appear in the browser.

6c. Verify

Open `https://yourdomain.com` — you should see the PathForge landing page with a green padlock.

Step 7 — Update Google OAuth Redirect URIs

Your Google OAuth credentials were configured for `localhost`. Update them for production.

1. Go to https://console.cloud.google.com → **APIs & Services → Credentials**
2. Click your OAuth 2.0 Client ID
3. Under **Authorised redirect URIs**, add:

   ```
   https://yourdomain.com/api/auth/callback/google
   ```

4. Under **Authorised JavaScript origins**, add:

   ```
   https://yourdomain.com
   ```

5. Click **Save**

Step 8 — Seed the Database

Run the seed script against Neon from your local machine:

```
cd PathForgeBackend
DATABASE_URL="<your-neon-connection-string>" npx ts-node prisma/seed.ts
```

If the seed script uses `npm run seed`, run:

```
cd PathForgeBackend
DATABASE_URL="<your-neon-connection-string>" npm run seed
```

This populates test users (`amina@pathforge.test`, `brian@pathforge.test`, etc.) and any other reference data.

Step 9 — Smoke Test Checklist

After all steps are complete, verify the following:

- [ ] `https://yourdomain.com` loads the PathForge landing page
- [ ] `https://yourdomain.com` shows a **green padlock** (SSL)
- [ ] `https://yourdomain.com/dashboard` redirects to login (auth guard working)
- [ ] Sign in with `amina@pathforge.test` / `Test@1234` succeeds and reaches the dashboard
- [ ] Google OAuth login works end-to-end

- [ ] Skill Gap page shows results for the **correct selected career** (not always "Data Scientist")
- [ ] Universities page career selector changes the matched programs
- [ ] Progress page shows actual skill scores
- [ ] `https://pathforge-backend.up.railway.app/health` returns `OK`
- [ ] `https://pathforge-ml.up.railway.app/` returns the Flask health JSON

Troubleshooting

"Application failed to respond" on Railway

- Check the **Deploy Logs** in Railway dashboard
- Confirm the `Procfile` is in the correct root directory
- For the Express backend, ensure `npm run build` succeeds (TypeScript compilation)
- For Flask, ensure `requirements.txt` is present in `Path-Forge API/`

CORS errors in the browser

- Confirm `FRONTEND_URL` in Railway matches your exact domain (with `https://`, no trailing slash)
- Confirm `NEXT_PUBLIC_API_BASE_URL` in Vercel matches your Railway backend URL

`NEXTAUTH_SECRET` mismatch

- The Express backend validates session tokens signed by NextAuth. Both services **must** use the identical `NEXTAUTH_SECRET` value. Copy-paste carefully.

404 on `/api/auth/callback/google`

- `NEXTAUTH_URL` must be set to your production domain in Vercel
- The redirect URI in Google Cloud Console must exactly match `https://yourdomain.com/api/auth/callback/google`

Prisma migration errors on Railway

- Railway runs `npx prisma migrate deploy` (not `migrate dev`) — this applies existing migration files only
- If you see "migration table not found", run `migrate deploy` manually via the Neon console or your local machine with the Neon `DATABASE_URL`

Neon SSL error

- Ensure your `DATABASE_URL` ends with `?sslmode=require`
- Prisma requires SSL for Neon; local `postgres://localhost` URLs do not have this suffix

Domain not resolving after 30 minutes

- Use https://dnschecker.org to check propagation for your A record
- Confirm you deleted name.com's default parking A records before adding Vercel's

---

Generated for PathForge Career Intelligence System — v2.0